Examen I

Desarrollo full stack: API REST en Laravel y cliente web con Vite

Fecha	Horario	Valor	Modalidad
Martes ___ de octubre de 2026	8:00 a. m. a 12:00 md (4 h)	20 % · 100 puntos	Individual, en laboratorio

Nombre completo	Carné	Usuario de GitHub

Docente: Lic. Alonso Chavarría Cubero

Estructura del examen

Parte	Horario	Puntos	Peso en la nota
A · Práctica: back-end Laravel (B1–B6)	8:00 – 11:00	36	7,2 %
A · Práctica: front-end Vite (F1–F5)	8:00 – 11:00	24	4,8 %
B · Comprensión del código propio (P1–P5)	11:00 – 12:00	40	8 %
Total		**100**	**20 %**

Parte A · Práctica (60 pts). Usted completa el repositorio base. Puede consultar cualquier material, incluidas herramientas de inteligencia artificial.

Parte B · Comprensión (40 pts). Se entrega impresa a las 11:00, con la computadora cerrada y el celular guardado. Las preguntas se refieren al código de **su** repositorio tal como quedó a las 11:00: cómo funciona, por qué está donde está y qué pasaría si cambiara. Una respuesta genérica, que no corresponda a su código, recibe como máximo la mitad del puntaje.

Instrucciones generales

1. El examen es **individual**. No se permite comunicarse con otras personas.
2. Clone el repositorio que se le asignó en GitHub. Contiene dos carpetas: `backend/` y `frontend/`.
3. Complete **únicamente** los archivos marcados con `TODO`. No modifique los archivos marcados **YA IMPLEMENTADO**, `routes/api.php` ni las migraciones.

4. Haga commit y push con frecuencia. Se califica el **último commit enviado antes de las 11:00 a. m.**; el historial de commits forma parte de la evidencia de autoría.
5. El front-end se califica contra su propio back-end. Si este no responde, se califica contra la API simulada incluida en el repositorio, de modo que una parte no arrastra a la otra.
6. **Usted responde por cada línea de su repositorio**, la haya escrito usted o una herramienta. Lea y entienda el código antes de enviarlo: la Parte B lo evalúa.

Contexto

El laboratorio de la carrera necesita controlar el préstamo de sus equipos (laptops, proyectores, kits de electrónica). Cada **equipo** pertenece a una **categoría** y está en uno de tres estados: `disponible`, `prestado` o `mantenimiento`. El repositorio ya incluye la base de datos con 25 equipos sembrados, la autenticación por token con Sanctum, las rutas y la estructura del cliente web.

Reglas de negocio

Regla	Descripción	Mensaje exacto
R1	No se puede eliminar un equipo prestado.	No se puede eliminar un equipo prestado.
R2	Solo se puede prestar un equipo disponible.	No se puede prestar un equipo en mantenimiento. · El equipo ya se encuentra prestado.

Una violación de cualquiera de estas reglas responde **409 Conflict** con el cuerpo `{ "mensaje": "…", "codigo": "REGLA_NEGOCIO" }`.

Puesta en marcha (≈ 15 min)

Requisitos: PHP 8.4 o superior, Composer, Node.js 22 y Git.

```
# Back-end  → http://localhost:8000/api
cd backend
composer run setup      # instala, crea .env, migra y siembra
php artisan serve
php artisan test        # 4 pruebas pasan, 3 quedan pendientes

# Front-end → http://localhost:5173   (otra terminal)
cd frontend
npm install
cp .env.example .env
npm run dev

# API simulada (opcional) → http://localhost:3001/api
npm run mock            # y en .env: VITE_API_URL=http://localhost:3001/api
```

Credenciales: `docente@ucr.ac.cr` · `Examen2026!`

Contrato de la API

Todas las rutas, salvo `POST /api/login`, exigen el encabezado `Authorization: Bearer <token>`.

Método	Ruta	Respuestas
GET	`/api/equipos?page=&per_page=&q=&estado=`	200 con `data`, `links` y `meta`. 10 por página por omisión, tope de 50.
GET	`/api/equipos/{id}`	200 · 404
POST	`/api/equipos`	201 con encabezado `Location` · 422
PUT	`/api/equipos/{id}`	200 · 404 · 422
DELETE	`/api/equipos/{id}`	204 · 404 · 409 (R1)
POST	`/api/equipos/{id}/prestamo`	200 · 404 · 409 (R2)
GET	`/api/categorias`	200 con `data: [{ id, nombre }]` (ya implementado)

Representación de un equipo (`EquipoResource`). El front-end depende de esta forma exacta:

```
{
  "id": 1,
  "codigo": "EQ-0001",
  "nombre": "Laptop Dell Latitude 5440",
  "estado": "disponible",
  "fecha_adquisicion": "2026-09-15",
  "categoria": { "id": 1, "nombre": "Cómputo" }
}
```

Parte A · Back-end Laravel (36 puntos)

B1 · Validación — app/Http/Requests/GuardarEquipoRequest.php (6 pts)

Se usa en `store()` y `update()`. Mensajes en español asociados a cada campo.

Campo	Reglas
`codigo`	Obligatorio, texto, máximo 20, formato `EQ-0000` y único. Al actualizar, ignora el equipo actual.
`nombre`	Obligatorio, texto, entre 3 y 120 caracteres.
`categoria_id`	Obligatorio, entero, debe existir en `categorias`.
`estado`	Opcional. Si viene, solo `disponible` o `mantenimiento`.
`fecha_adquisicion`	Obligatoria, fecha válida, no futura.
`costo_interno`	Obligatorio, numérico, mayor o igual a 0.

B2 · Capa de negocio — app/Services/EquipoService.php (9 pts)

- `listar($q, $estado, $porPagina)`: paginado, ordenado por código, con la categoría cargada de forma anticipada (sin N+1). `q` busca en código o nombre; `estado` filtra por estado exacto.
- `crear`, `actualizar`: devuelven el equipo con la categoría cargada.
- `eliminar` aplica **R1**; `prestar` aplica **R2** y cambia el estado a `prestado`.

- El servicio **no conoce HTTP**: lanza `ReglaNegocioException`; nunca usa `abort()`, `request()` ni `response()`.

B3 · Controlador — app/Http/Controllers/Api/EquipoController.php (6 pts)

Controlador delgado: ningún método supera 15 líneas, ninguno contiene reglas de negocio y todos responden con `EquipoResource`. Debe cumplir los códigos del contrato, incluido el encabezado `Location` en la creación y el tope de 50 en `per_page`.

B4 · Recurso — app/Http/Resources/EquipoResource.php (5 pts)

Forma exacta del contrato. `costo_interno`, `created_at` y `updated_at` no deben aparecer. Reutilice `CategoriaResource`.

B5 · Errores centralizados — bootstrap/app.php (4 pts)

Traduzca `ReglaNegocioException` a 409 con `{ mensaje, codigo: "REGLA_NEGOCIO" }` y evite que se registre en la bitácora.

B6 · Pruebas — tests/Feature/EquipoApiTest.php (6 pts)

Implemente las tres pruebas indicadas (201 con `Location`, 422 con errores por campo, 409 al eliminar un equipo prestado). Cada una debe verificar el código de estado **y** algo más.

Parte A · Front-end Vite (24 puntos)

F1 · Cliente HTTP — src/modules/api.js y equipos.js (6 pts)

Un único `request(ruta, opciones)` que envía `Accept: application/json` y el token en `Authorization`, serializa el cuerpo a JSON, cancela la petición a los 8 s con `AbortController`, devuelve `null` en 204, lanza `ErrorApi` si `response.ok` es falso (y en 401 olvida la sesión y recarga). Complete `listarEquipos`, `crearEquipo` y `eliminarEquipo` usando `request()`.

F2 · Listado y paginación — src/components/listado.js (6 pts)

Clone `#tpl-equipo` por cada equipo y llene sus datos **solo con `textContent`**. Marque `data-estado` y `data-id`, muestre el resumen (“Mostrando 1–6 de 25”) y la página actual, deshabilite Anterior/Siguiente en los extremos, use `aria-busy` durante la carga y muestre un estado vacío.

F3 · Formulario — src/components/formulario.js (6 pts)

Cargue las categorías en el `select`. Al enviar: limpie errores previos, convierta `categoria_id` y `costo_interno` a número, deshabilite el botón con “Guardando…”, muestre cada error 422 en su `<p data-error>` con `aria-invalid="true"` en el campo y, al guardar, limpie el formulario y recargue la lista.

F4 · Eliminación — src/components/listado.js (3 pts)

Con **delegación de eventos** sobre la lista: confirme, elimine, recargue la página (o la anterior si quedó vacía) y, ante un 409, muestre el mensaje que envía la API.

F5 · Maquetación — src/styles/layout.css (3 pts)

Mobile first: una columna en móvil; desde 900 px, Grid de 320 px + 1fr con el formulario `sticky`. La lista usa `auto-fill` con `minmax(220px, 1fr)`. Barras con Flexbox. Sin desplazamiento horizontal a 360 px.

Rúbrica de evaluación

Parte A · Práctica (60 puntos)

Id	Criterio	Pts	Logrado (puntaje completo)	Parcial	No logrado (0)
B1	Validación (Form Request)	6	Las seis reglas completas: formato EQ-0000, `unique` que ignora el equipo actual, `exists`, `estado` limitado a disponible/mantenimiento, fecha no futura, costo ≥ 0. Mensajes en español por campo.	Faltan reglas, el `unique` falla al actualizar o hay mensajes en inglés. (3 pts)	No hay Form Request o la validación se hace en el controlador.
B2	Servicio y reglas de negocio	9	R1 y R2 (con sus dos mensajes) en el servicio, con `ReglaNegocioException`. Listado con `with('categoria')` y filtros `q` y `estado`. El servicio no usa `request()`, `response()` ni `abort()`.	Solo una regla, usa `abort()` en el servicio, o el listado tiene N+1 o no filtra. (5 pts)	Las reglas están en el controlador o no existen.
B3	Controlador y códigos de estado	6	Métodos de 15 líneas o menos que solo delegan. 200, 201 con `Location`, 204, 404 y 422 correctos. Tope `per_page` de 50 impuesto por el servidor.	Algún código incorrecto, falta `Location` o falta el tope. (3 pts)	Devuelve 200 con el error en el cuerpo, o hay lógica de negocio en el controlador.
B4	Recurso de API y paginación	5	`EquipoResource` con los seis campos exactos del contrato, `categoria` con `whenLoaded` y `CategoriaResource`, fecha `Y-m-d`. Listado con `data`, `links` y `meta`.	Forma distinta al contrato, expone marcas de tiempo o la fecha sale en otro formato. (3 pts)	Devuelve el modelo directo o expone `costo_interno`.
B5	Manejo centralizado de errores	4	`ReglaNegocioException` traducida a 409 en `bootstrap/app.php` con `{ mensaje, codigo }` y excluida de la bitácora.	Traducida a 409 pero con otro cuerpo, o sin `dontReport`. (2 pts)	Manejo con try/catch disperso, o la respuesta expone una traza de pila.
B6	Pruebas automatizadas (Pest)	6	Las tres pruebas pasan y cada una verifica algo más que el código de estado (encabezado, estructura, mensaje o estado de la base).	Una o dos pruebas pasan, o solo verifican el código de estado. (3 pts)	No hay pruebas o ninguna pasa.
F1	Cliente HTTP (api.js y equipos.js)	6	`request()` único: Bearer en `Authorization`, JSON automático, `AbortController`, 204 → null, `response.ok` verificado y `ErrorApi` lanzado; 401 limpia la sesión. Las tres funciones usan `request()`.	No verifica `response.ok`, no maneja 204 o falta el tiempo máximo. (3 pts)	`fetch` disperso en los componentes o el token viaja en la URL.

Id	Criterio	Pts	Logrado (puntaje completo)	Parcial	No logrado (0)
F2	Listado y paginación	6	Tarjetas con `<template>` y `textContent`, `data-estado` y `data-id`, resumen y página actual, botones deshabilitados en los extremos, `aria-busy` durante la carga, estado vacío.	Lista sin paginación funcional o sin resumen. (3 pts)	Usa `innerHTML` con datos de la API o no lista.
F3	Formulario de registro	6	Select de categorías cargado, números convertidos, botón deshabilitado con "Guardando…", errores 422 junto a cada campo con `aria-invalid`, limpieza previa, `reset` y recarga al guardar.	Muestra un error genérico en lugar de los errores por campo, o no restaura el botón. (3 pts)	No envía o no maneja errores.
F4	Eliminación con regla de negocio	3	Delegación de eventos en la lista, `confirm()`, 204 recarga la página (retrocede si quedó vacía) y 409 muestra el mensaje de la API.	Solo funciona el caso feliz, o un listener por tarjeta. (2 pts)	No elimina.
F5	Maquetación responsiva	3	Grid de una columna en móvil y 320px + 1fr desde 900px, formulario `sticky`, lista con `auto-fill`/`minmax(220px, 1fr)`, barras con Flexbox, sin desplazamiento horizontal a 360px.	Responsivo parcial o usa media queries donde se pedía `auto-fill`. (2 pts)	Se rompe en móvil o no hay maquetación.

Si una parte no levanta, el criterio se evalúa leyendo el código y su puntaje máximo es el de **Parcial**.

Parte B · Comprensión (40 puntos)

Cinco preguntas sobre su propio repositorio. Cada respuesta se califica en tres niveles:

Nivel	Descripción
Completo	Técnicamente correcta y coherente con el código de su repositorio: nombra archivos, funciones o líneas concretas.
Parcial	Correcta en lo general pero incompleta, o genérica, sin relación verificable con su código (máximo la mitad del puntaje).
Insuficiente	Incorrecta, en blanco, o contradice lo que hace su código.